

บทที่ 8 การจัดกลุ่ม (Clustering)

วัตถุประสงค์การเรียนรู้

1. อธิบายความหมาย หลักการ และประเภทของ Clustering ในฐานงาน Unsupervised Learning รวมถึงเป้าหมายและการประยุกต์ใช้งาน
2. อธิบายทฤษฎี K-Means และ K-Medoids รวมถึง Within-Cluster Variance, Medoid, การเลือกจำนวน Cluster K และ Gap Statistic
3. ใช้ K-Means และ K-Medoids สำหรับ Partition Clustering และเปรียบเทียบความทนทานต่อ Outlier พร้อมประเมินด้วย Silhouette Score และ Adjusted Rand Score
4. อธิบายทฤษฎี Hierarchical Clustering รวมถึง Agglomerative (Bottom-Up), Divisive (Top-Down), Linkage Methods (Single, Complete, Average) และการแสดงผลด้วย Dendrogram
5. ประยุกต์ใช้ Agglomerative Hierarchical Clustering และเลือกจำนวน Cluster ที่เหมาะสมโดยใช้ Silhouette Score
6. อธิบายทฤษฎี DBSCAN รวมถึงนิยาม Core Point, Border Point, Noise Point, Density-Reachability และความแตกต่างกับ OPTICS พร้อมดำเนินการ DBSCAN โดยปรับค่า eps และ min_samples
7. อธิบายทฤษฎีและใช้งาน Grid-Based Clustering ได้แก่ STING (Statistical Information Grid) และ CLIQUE (Subspace Clustering) รวมถึงเปรียบเทียบกับ OPTICS บนชุดข้อมูล Two Moons

8. อธิบายทฤษฎี PCA รวมถึง SVD, Explained Variance Ratio, Scree Plot และข้อจำกัดเชิง Linear และลดมิติข้อมูลด้วย PCA ทั้งแบบกำหนดจำนวน Component และแบบกำหนดสัดส่วน Variance (0.80, 0.95)
9. เปรียบเทียบประสิทธิภาพของ Partition (K-Means), Hierarchical (Agglomerative) และ Density-Based (DBSCAN) Clustering โดยใช้ Silhouette Score บนชุดข้อมูล Penguins

8.1 บทนำ

Clustering เป็นงาน Unsupervised Learning ที่อัลกอริทึมจัดกลุ่มชุดของวัตถุเพื่อให้วัตถุในกลุ่มเดียวกัน (Cluster) มีความคล้ายคลึงกันมากกว่าวัตถุในกลุ่มอื่น เป้าหมายของ Clustering คือการค้นพบการจัดกลุ่มหรือโครงสร้างที่แฝงอยู่ในข้อมูล

วิธีการ Clustering ที่นิยมใช้ใน Scikit-Learn มีดังนี้:

- K-Means: วิธีการที่แบ่งชุดข้อมูลออกเป็น k กลุ่ม โดยแต่ละกลุ่มถูกกำหนดโดยค่าเฉลี่ยของจุดข้อมูลที่อยู่ในกลุ่มนั้น
- Hierarchical Clustering: วิธีการที่สร้างลำดับขั้นของกลุ่ม โดยแต่ละกลุ่มจะถูกแบ่งออกเป็นกลุ่มย่อยหรือรวมกับกลุ่มอื่น
- DBSCAN (Density-Based Spatial Clustering of Applications with Noise): วิธีการ Clustering ตามความหนาแน่น จัดกลุ่มจุดที่อยู่ใกล้กัน และทำเครื่องหมายจุดที่อยู่โดดเดี่ยวเป็น Outlier
- Gaussian Mixture Model: โมเดลความน่าจะเป็นที่แสดงชุดข้อมูลในรูปแบบของการผสม Gaussian Distribution

8.2 Partition Clustering

Partition Clustering เป็นเทคนิคพื้นฐานใน Unsupervised Machine Learning ที่ใช้จัดกลุ่มจุดข้อมูลที่คล้ายกันเข้าเป็น Cluster หรือ Partition ส่วนนี้แนะนำวิธีการ Partition Clustering ซึ่งรวมถึง K-Means และ K-Medoids โดยใช้แพ็คเกจ Scikit-learn

Partition Clustering มีเป้าหมายเพื่อค้นพบการจัดกลุ่มตามธรรมชาติหรือ Cluster ภายใน ชุด ข้อมูล โดย จุด ข้อมูล ภายใน Cluster เดียวกันมีความคล้ายคลึงกันมากกว่าจุดในกลุ่มอื่น ใช้กันอย่างแพร่หลายในหลายสาขา รวมถึงการแบ่งส่วนลูกค้า การประมวลผลภาพ และการตรวจจับความผิดปกติ

8.2.1 ทฤษฎี Partition Clustering

Silhouette Coefficient

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

โดยที่:

$s(i)$ = Silhouette Coefficient ของจุดข้อมูล i มีค่าระหว่าง -1 ถึง 1

$a(i)$ = ระยะทางเฉลี่ยจากจุด i ไปยังจุดอื่นๆ ในกลุ่มเดียวกัน (ยิ่งน้อยยิ่งดี)

$b(i)$ = ระยะทางเฉลี่ยน้อยที่สุดจากจุด i ไปยังกลุ่มที่ใกล้เคียงที่สุดที่ไม่ใช่กลุ่มของตัวเอง

การตีความ: $s(i) \approx 1$ = จัดกลุ่มถูกต้อง, $s(i) \approx 0$ = อยู่บนขอบแดนระหว่างกลุ่ม, $s(i) <$

0 = อาจจัดกลุ่มผิด

Partition Clustering เป็นวิธีการจัดกลุ่มพื้นฐานที่กำหนดให้ข้อมูลแต่ละจุดอยู่ใน Cluster เดียวเท่านั้น โดยมีเป้าหมายเพื่อลดความแปรปรวนภายใน Cluster (Within-Cluster Variance) ให้น้อยที่สุด Tan et al. (2006) อธิบายว่า Partition Clustering พยายามค้นพบการจัดกลุ่มตามธรรมชาติ (Natural Groupings) ในชุดข้อมูล โดยจุดข้อมูลภายใน Cluster เดียวกันมีความคล้ายคลึงกันมากกว่าจุดข้อมูลในกลุ่มอื่น

K-Means Algorithm

K-Means เป็นอัลกอริทึม Partition Clustering ที่ได้รับความนิยมสูงสุด โดย Hastie, Tibshirani และ Friedman (2009) อธิบายว่า K-Means ออกแบบมาสำหรับข้อมูลเชิงปริมาณ (Quantitative Variables) โดยใช้ Squared Euclidean Distance เป็นตัววัดความไม่คล้ายคลึง ฟังก์ชันวัตถุประสงค์ (Objective Function) ที่ต้องการลดค่าคือ Within-Cluster Point Scatter:

$$\begin{aligned} W(C) &= \frac{1}{2} \sum_{k=1}^K \sum_{C(i)=k} \sum_{C(i')=k} d(x_i, x_{i'}) \\ &= \sum_{k=1}^K N_k \sum_{C(i)=k} \|x_i - \bar{x}_k\|^2 \end{aligned}$$

โดยที่ $\bar{x}_k$ คือ Mean Vector ของ Cluster k และ N_k คือจำนวนจุดข้อมูลใน Cluster k อัลกอริทึม K-Means ทำงานแบบวนซ้ำ (Iterative) โดย (1) กำหนด Cluster ของแต่ละจุดโดยหา Centroid ที่ใกล้ที่สุด $C(i) = \arg \min_k \|x_i - m_k\|^2$ (2) อัปเดต Centroid ให้เป็นค่าเฉลี่ยของจุดใน Cluster นั้น และ (3) วนซ้ำจนกว่าการมอบหมาย Cluster จะไม่เปลี่ยนแปลง

K-Medoids Algorithm

K-Medoids เป็นส่วนขยายของ K-Means ที่มีความทนทานต่อ Outlier มากกว่า Hastie et al. (2009) อธิบายว่า K-Medoids แตกต่างจาก K-Means ตรงที่ใช้จุดข้อมูลจริงในชุดข้อมูลเป็นตัวแทน Cluster (Medoid) แทนที่จะเป็นค่าเฉลี่ย ทำให้สามารถใช้กับการวัดความไม่คล้ายคลึง (Dissimilarity) ที่กำหนดเองได้ อัลกอริทึมทำงานโดย: (1) หา Medoid i^*k ที่ลด Total Distance ภายใน Cluster: $i^*k = \arg \min \sum D(x_i, x_{i'})$ (2) มอบหมายแต่ละจุดให้กับ Medoid ที่ใกล้ที่สุด: $C(i) = \arg \min D(x_i, m_k)$ โดยข้อเสียของ K-Medoids คือมีความซับซ้อนในการคำนวณ $O(Nk^2)$ ต่อ Cluster ซึ่งสูงกว่า K-Means

การเลือกจำนวน Cluster K

Tan et al. (2006) แนะนำว่าการเลือก K ที่เหมาะสมเป็นปัญหาสำคัญใน Partition Clustering วิธีที่นิยมคือการพิจารณา Within-Cluster Dissimilarity (WK) เป็นฟังก์ชันของ K โดยทั่วไป WK จะลดลงเมื่อ K เพิ่มขึ้น การหา “จุดพับ” (Kink) ในกราฟจะบ่งบอกถึงจำนวน Cluster ที่เหมาะสม Hastie et al. (2009) เสนอ Gap Statistic ซึ่งเปรียบเทียบค่า $\log(WK)$ กับค่าที่คาดหวังจากข้อมูลที่กระจายแบบ Uniform เพื่อประมาณค่า K^* ที่ดีที่สุด

8.2.2 แบบฝึกหัด

เราเรียนรู้ทั้ง K-Means และ K-Medoids เป็นวิธี Clustering พื้นฐาน มาทดลองใช้งานและสังเกตความแตกต่างกัน

8.2.2.1 การเตรียม Package

- K-Means รวมอยู่ใน sklearn package สามารถ import ได้โดยตรง
- K-Medoids รวมอยู่ใน sklearn extra package ต้องติดตั้งก่อนแล้วจึง import
- ต้องใช้ NumPy และ Pandas เพื่อสร้างชุดข้อมูลจำลอง
- ต้องใช้ Seaborn เพื่อแสดงผลลัพธ์

```
!pip install scikit-learn-extra
```

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn_extra.cluster import KMedoids
import matplotlib.pyplot as plt
import seaborn as sns
```

8.2.2.2 ทดลองที่ 1: สร้างชุดข้อมูลเรียบง่าย

เพื่อวัตถุประสงค์ในการแสดงภาพ สร้างจุดข้อมูล 3 กลุ่ม กลุ่มละ 100 จุด มีสองมิติ กระจายแบบ Normal Distribution ตามค่าเฉลี่ยและ std ที่กำหนด:

```
c1 = np.random.normal(5, 3, (100, 2))
c2 = np.random.normal(15, 5, (100, 2))
c3 = np.random.normal(-5, 2, (100, 2))
d = np.concatenate((c1, c2, c3), axis=0)
d.shape # (300, 2)

sns.scatterplot(data=d, x=d[:,0], y=d[:,1])
```

ฝึกโมเดลด้วย K-Means:

```
kmeans = KMeans(n_clusters=3,
random_state=0).fit(d)
sns.scatterplot(data=d, x=d[:,0], y=d[:,1],
hue=kmeans.predict(d))
```

ฝึกโมเดลด้วย K-Medoids:

```
kmedoids = KMedoids(n_clusters=3,
random_state=0).fit(d)
sns.scatterplot(data=d, x=d[:,0], y=d[:,1],
hue=kmedoids.predict(d))
```

สรุป: K-Means และ K-Medoids ให้ผลลัพธ์ที่ดีพอๆ กันสำหรับชุดข้อมูลนี้

8.2.2.3 ทดลองที่ 2: เพิ่ม Outlier หนึ่งจุด

สร้าง Cluster เดิมพร้อม Outlier ที่อยู่ห่างไกลจากจุดข้อมูลอื่น (100, 100):

```
c1 = np.random.normal(5, 3, (100, 2))
c2 = np.random.normal(15, 5, (100, 2))
c3 = np.random.normal(-5, 2, (100, 2))
outlier = np.array([[100, 100]])
d = np.concatenate((c1, c2, c3, outlier),
axis=0)
```

```
kmeans = KMeans(n_clusters=3,
random_state=0).fit(d)
kmedoids = KMedoids(n_clusters=3,
random_state=0).fit(d)
```

สรุป: K-Medoids ทำงานได้ดีแม้มี Outlier แต่ K-Means ล้มเหลว เพราะ K-Means ใช้ค่าเฉลี่ยซึ่งถูกดึงโดย Outlier

8.2.2.4 ทดลองที่ 3: สร้างชุดข้อมูลสมจริงด้วย Pandas

สร้างชุดข้อมูลโดยใช้ Pandas DataFrame สำหรับการจัดการที่สะดวกยิ่งขึ้น:

```
df = pd.DataFrame({'x': np.random.normal(5, 3,
100), 'y': np.random.normal(-2, 2, 100)})
df = pd.concat([df, pd.DataFrame({'x':
np.random.normal(15, 2, 100), 'y':
np.random.normal(22, 2, 100)})],
ignore_index=True)
df = pd.concat([df, pd.DataFrame({'x':
np.random.normal(-5, 3, 100), 'y':
np.random.normal(8, 2, 100)})],
ignore_index=True)

kmeans = KMeans(n_clusters=3,
random_state=0).fit(df)
kmedoids = KMedoids(n_clusters=3,
random_state=0).fit(df)
```

สรุป: K-Means และ K-Medoids ให้ผลลัพธ์ที่ดีพอๆ กัน

8.2.2.5 ทดลองที่ 4: เพิ่ม Outlier

เพิ่ม Outlier (100, 100) เข้าในชุดข้อมูล:

```
df = pd.concat([df, pd.DataFrame({'x': [100],
'y': [100]})], ignore_index=True)
```

สรุป: K-Medoids ทำงานได้ดีแม้มี Outlier แต่ K-Means ล้มเหลว

8.2.2.6 ทดลองที่ 5: เพิ่ม Outlier สองจุด

เพิ่ม Outlier สองจุดที่ระยะห่างมากขึ้น:

```
df = pd.concat([df, pd.DataFrame({'x': [200],  
    'y': [200]})], ignore_index=True)  
df = pd.concat([df, pd.DataFrame({'x': [-200],  
    'y': [-200]})], ignore_index=True)
```

สรุป: K-Medoids ทำงานได้ดีแม้มี Outlier มากขึ้น แต่ K-Means ยังคงล้มเหลว

8.2.3 กรณีศึกษา

ดำเนินการวิเคราะห์ Clustering โดยใช้ชุดข้อมูล Iris เพื่อแสดงอัลกอริทึม K-Means และ K-Medoids และเปรียบเทียบผลลัพธ์ตามประสิทธิภาพและ Cluster ที่ก่อตัวขึ้น

8.2.3.1 การตั้งค่า

โหลดและสำรวจชุดข้อมูล Iris:

```
!pip install scikit-learn-extra
```

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
from sklearn.datasets import load_iris  
from sklearn.preprocessing import StandardScaler  
from sklearn.cluster import KMeans  
from sklearn_extra.cluster import KMedoids  
from sklearn.metrics import silhouette_score,  
adjusted_rand_score  
  
data = load_iris()  
X, y = data.data, data.target
```



```
df = pd.DataFrame(data=np.c_[X, y],
                  columns=data.feature_names + ['target'])
print(df.head())
print(df.describe())
print(df.info())
```

8.2.3.2 การเตรียมข้อมูล

ก่อนใช้อัลกอริทึม Clustering ต้องทำให้ Feature อยู่ในมาตรฐานเดียวกันด้วย StandardScaler:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

8.2.3.3 K-Means Clustering

```
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_scaled)
kmeans_labels = kmeans.labels_
kmeans_silhouette = silhouette_score(X_scaled,
kmeans_labels)
kmeans_ari = adjusted_rand_score(y,
kmeans_labels)
print(f'K-means Silhouette Score:
{kmeans_silhouette:.4f}') # 0.4599
print(f'K-means ARI Score: {kmeans_ari:.4f}') #
0.6201
```

8.2.3.4 K-Medoids Clustering

```
kmedoids = KMedoids(n_clusters=3,
random_state=42)
kmedoids.fit(X_scaled)
kmedoids_labels = kmedoids.labels_
kmedoids_silhouette =
silhouette_score(X_scaled, kmedoids_labels)
kmedoids_ari = adjusted_rand_score(y,
kmedoids_labels)
print(f'K-medoids Silhouette Score:
{kmedoids_silhouette:.4f}') # 0.4590
```

```
print(f'K-medoids          ARI          Score:
      {kmedoids_ari:.4f}') # 0.6312
```

8.2.3.5 เปรียบเทียบ K-Means และ K-Medoids

แสดงภาพผลการ Cluster ของทั้งสองวิธีพร้อม Centroids/Medoids:

```
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.scatter(X_scaled[:,0], X_scaled[:,1],
            c=kmeans_labels, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[0],
            kmeans.cluster_centers_[1],
            marker='X', s=200, c='red', label='Centroids')
plt.title('K-means Clustering')
plt.subplot(1, 2, 2)
plt.scatter(X_scaled[:,0], X_scaled[:,1],
            c=kmedoids_labels, cmap='viridis')
plt.scatter(kmedoids.cluster_centers_[0],
            kmedoids.cluster_centers_[1],
            marker='X', s=200, c='red', label='Medoids')
plt.title('K-medoids Clustering')
plt.tight_layout()
plt.show()
```

8.3 Hierarchical Clustering

Hierarchical Clustering เป็นเทคนิคที่ทรงพลังใน Unsupervised Machine Learning ใช้ค้นพบโครงสร้างแบบลำดับชั้นและการจัดกลุ่มตามธรรมชาติภายในชุดข้อมูล โดยสร้างโครงสร้างคล้ายต้นไม้ (Dendrogram) ที่แสดงการจัดกลุ่มซ้อนกันของข้อมูล

ส่วนนี้แนะนำ Hierarchical Clustering โดยใช้แพ็คเกจ Scikit-learn ซึ่งมีเครื่องมือสำหรับ Agglomerative Hierarchical Clustering พร้อมปรับแต่ง Linkage Method ต่างๆ เช่น Ward, Complete, Average

8.3.1 ทฤษฎี Hierarchical Clustering

Single Linkage

$$d(C_i, C_j) = \min \{ d(x, y) : x \in C_i, y \in C_j \}$$

Single Linkage (MIN):

ระยะทางระหว่างกลุ่มคือระยะทางน้อยที่สุดระหว่างสมาชิกสองกลุ่ม

$$d(C_i, C_j) = \min \{ d(x, y) : x \in C_i, y \in C_j \}$$

ผลลัพธ์: มีแนวโน้มสร้างกลุ่มยาวต่อเนื่องกัน (Chaining Effect)

เหมาะกับข้อมูลที่มีรูปร่างไม่สม่ำเสมอ

Complete Linkage

$$d(C_i, C_j) = \max \{ d(x, y) : x \in C_i, y \in C_j \}$$

Complete Linkage (MAX):

ระยะทางระหว่างกลุ่มคือระยะทางมากที่สุดระหว่างสมาชิกสองกลุ่ม

$$d(C_i, C_j) = \max \{ d(x, y) : x \in C_i, y \in C_j \}$$

ผลลัพธ์: สร้างกลุ่มที่มีขนาดกะทัดรัด (Compact) และสม่ำเสมอ แต่ไวต่อค่าผิดปกติ

Average Linkage

$$d(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i, y \in C_j} d(x, y)$$

Average Linkage (UPGMA):

ระยะทางระหว่างกลุ่มคือค่าเฉลี่ยของระยะทางทุกคู่ข้ามกลุ่ม

$$d(C_i, C_j) = (1 / |C_i||C_j|) \cdot \sum \sum d(x, y)$$

ผลลัพธ์: ประนีประนอมระหว่าง Single และ Complete Linkage ลดผลกระทบจาก

Chaining Effect และค่าผิดปกติ

Hierarchical Clustering เป็นวิธีการที่ไม่ต้องกำหนดจำนวน Cluster ล่วงหน้า แต่สร้างลำดับชั้นของ Cluster แทน Hastie et al. (2009) อธิบายว่า Hierarchical Clustering แบ่งออกเป็นสองกลุ่มหลักตามทิศทางการสร้างลำดับชั้น ได้แก่ Agglomerative (Bottom-Up) และ Divisive (Top-Down)

Agglomerative Hierarchical Clustering

Agglomerative Clustering เริ่มต้นโดยถือว่าแต่ละจุดข้อมูลเป็น Cluster ของตัวเอง (Singleton Cluster) แล้วรวม Cluster ที่มีความไม่คล้ายคลึง (Intergroup Dissimilarity) น้อยที่สุดเข้าด้วยกันทีละคู่ในแต่ละขั้น Tan et al. (2006) อธิบายว่ากระบวนการนี้ผลิต $N-1$ ระดับในลำดับชั้น และสามารถแสดงผลในรูปแบบ Dendrogram ซึ่งเป็นโครงสร้างคล้ายต้นไม้ที่แสดงลำดับการรวม Cluster ตาม Height ที่สัมพันธ์กับค่าความไม่คล้ายคลึงระหว่าง Cluster

Linkage Methods

วิธีการวัดความไม่คล้ายคลึงระหว่าง Cluster (Linkage Method) มีผลสำคัญต่อรูปร่างของ Cluster ที่ได้ Tan et al. (2006) อธิบาย Linkage Method หลัก 3 ชนิด ได้แก่ (1) Single Linkage: ใช้ระยะทางขั้นต่ำระหว่างจุดใดๆ ในสอง Cluster มีแนวโน้มสร้าง Cluster ยาวและเชื่อมต่อ (Chain-like) (2) Complete Linkage: ใช้ระยะทางสูงสุดระหว่างจุดใดๆ ในสอง Cluster มีแนวโน้มสร้าง Cluster กลมและขนาดใกล้เคียงกัน (3) Average Linkage: ใช้ค่าเฉลี่ยระยะทางระหว่างทุกคู่จุดข้อมูลใน Cluster ทั้งสอง เป็นวิธีประนีประนอมที่นิยมใช้

Divisive Hierarchical Clustering

Divisive Clustering เริ่มจาก Cluster เดียวที่มีข้อมูลทั้งหมด แล้วแบ่งซ้ำๆ ออกเป็น Sub-cluster Hastie et al. (2009) กล่าวว่า Divisive Method ที่หลีกเลี่ยงปัญหา Monotonicity ถูกเสนอโดย Macnaughton Smith et al. (1965) โดยเริ่มจากการหาจุดข้อมูลที่มีค่าเฉลี่ยความไม่คล้ายคลึงกับจุดอื่นสูงที่สุด

แล้วย้ายจุดดังกล่าวไปยัง Cluster ใหม่ จากนั้นย้ายจุดอื่นๆ ที่ใกล้ชิดกับ Cluster ใหม่มากกว่า Cluster เดิมไปเรื่อยๆ จนกว่าจะไม่มีจุดใดที่ควรย้าย

Silhouette Score

Silhouette Score เป็นตัววัดคุณภาพ Clustering ที่ Tan et al. (2006) แนะนำ โดยคำนวณจาก:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

โดยที่ $a(i)$ คือค่าเฉลี่ยระยะทางจากจุด i ไปยังจุดอื่นๆ ใน Cluster เดียวกัน และ $b(i)$ คือค่าเฉลี่ยระยะทางน้อยที่สุดจากจุด i ไปยัง Cluster อื่นๆ ที่ใกล้ที่สุด ค่า Silhouette Score อยู่ในช่วง $[-1, 1]$ ค่าใกล้ 1 หมายถึงจุดข้อมูลอยู่ใน Cluster ที่ถูกต้องดี ค่าใกล้ -1 หมายถึงจุดข้อมูลน่าจะอยู่ใน Cluster อื่น

8.3.2 แบบฝึกหัด

สร้างชุดข้อมูลจำลองและแสดง Agglomerative Hierarchical Clustering ด้วยจำนวน Cluster ต่างๆ เพื่อสังเกตผลลัพธ์และ Dendrogram

8.3.2.1 สร้างชุดข้อมูลจำลอง

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score

X, y = make_blobs(n_samples=100, centers=4,
                  n_features=2,
                  random_state=42, cluster_std=1.5)
```

```
df = pd.DataFrame(data=np.c_[X, y],
columns=['Feature1', 'Feature2', 'Cluster'])
print(df.head())
print(df.describe())
plt.scatter(df['Feature1'], df['Feature2'])
plt.show()
```

8.3.2.2 การเตรียมข้อมูล

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

8.3.2.3 Agglomerative Hierarchical Clustering

ทดสอบ Agglomerative Clustering ด้วยจำนวน Cluster ตั้งแต่ 2 ถึง 7:

```
for num_clusters in range(2, 8):
    agglomerative =
    AgglomerativeClustering(n_clusters=num_clusters
    )
    labels = agglomerative.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    print(f'Clusters: {num_clusters}, Silhouette:
    {score:.4f}')
```

ผลลัพธ์: 2 Cluster → 0.5268, 3 Cluster → 0.6894, 4 Cluster → 0.6966 (ดีที่สุด),
5 Cluster → 0.5915 — 4 Cluster สอดคล้องกับโครงสร้างข้อมูลจริง

8.3.3 กรณีศึกษา

ดำเนินการ Hierarchical Clustering โดยใช้ชุดข้อมูล Iris และแสดง Agglomerative Clustering ซึ่งเป็นวิธีที่นิยมใช้กันอย่างแพร่หลาย

8.3.3.1 การตั้งค่า

```
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

data = load_iris()
X, y = data.data, data.target
df = pd.DataFrame(data=np.c_[X, y],
                  columns=data.feature_names + ['target'])
print(df.head())
print(df.describe())
```

8.3.3.2 การเตรียมข้อมูล

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

8.3.3.3 Agglomerative Hierarchical Clustering

```
agglomerative = AgglomerativeClustering(n_clusters=3)
agglomerative.fit(X_scaled)
agglomerative_labels = agglomerative.labels_
print(f'Cluster Assignments: {agglomerative_labels}')
```

ผลลัพธ์: ได้ Cluster Assignments สำหรับ 150 ตัวอย่างในชุดข้อมูล Iris แบ่งเป็น 3 กลุ่ม

8.4 Density-Based Clustering

Density-Based Clustering เป็นเทคนิคที่ทรงพลังใน Unsupervised Machine Learning ใช้ค้นพบ Cluster ที่มีรูปร่างและขนาดหลากหลายตามความหนาแน่นของข้อมูล

ส่วนนี้ แนะนำ DBSCAN (Density-Based Spatial Clustering of Applications with Noise) ซึ่งจัดกลุ่มจุดตามความหนาแน่นในพื้นที่ Feature

DBSCAN มีประโยชน์อย่างยิ่งสำหรับค้นพบ Cluster ที่มีรูปร่างไม่สม่ำเสมอและจัดการกับ Noise ได้ดี สามารถปรับแต่งได้ด้วยพารามิเตอร์ epsilon (ระยะ Neighborhood) และ min_samples

8.4.1 ทฤษฎี Density-Based Clustering

Density-Based Clustering เป็นวิธีการจัดกลุ่มที่ใช้ความหนาแน่นของข้อมูล (Data Density) เป็นเกณฑ์หลักในการกำหนด Cluster Tan et al. (2006) อธิบายว่า Density-Based Clustering มีจุดเด่นที่สำคัญ คือ สามารถค้นพบ Cluster ที่มีรูปร่างไม่สม่ำเสมอ (Arbitrary Shape) ได้ดี และสามารถจัดการกับ Noise และ Outlier ได้อย่างมีประสิทธิภาพ ซึ่งเป็นข้อจำกัดของ K-Means และ Hierarchical Clustering

DBSCAN: แนวคิดหลัก

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) นิยาม Cluster เป็นบริเวณที่มีความหนาแน่นสูงในพื้นที่ Feature โดยใช้พารามิเตอร์สองตัว ได้แก่ Epsilon (ϵ): รัศมี ของ Neighborhood และ MinPts (min_samples): จำนวนจุดขั้นต่ำที่ต้องมีภายในรัศมี ϵ Tan et al. (2006) แบ่งจุดข้อมูลออกเป็น 3 ประเภท: (1) Core Point คือจุดที่มีจุดข้อมูลอย่างน้อย MinPts จุดอยู่ในรัศมี ϵ (2) Border Point คือจุดที่อยู่ใน Neighborhood ของ Core Point แต่ตัวเองไม่ใช่ Core Point (3) Noise Point คือจุดที่ไม่ใช่ทั้ง Core Point และ Border Point

กระบวนการทำงานของ DBSCAN

DBSCAN ทำงานโดยเริ่มจากจุด Core Point ใดๆ ที่ยังไม่ถูกมอบหมาย Cluster แล้วขยาย Cluster โดยรวมจุดทุกจุดที่ density-reachable จาก Core Point นั้น กล่าวคือ จุด q เป็น directly density-reachable จาก p ถ้า p เป็น Core Point และ q อยู่ในรัศมี ϵ ของ p จุด q เป็น density-reachable จาก p ถ้ามีเส้นทางของ directly density-

reachable จาก p ไปยัง q กระบวนการนี้ดำเนินต่อจนกว่าจะขยาย Cluster ไม่ได้อีก แล้วจึงเริ่มต้น Cluster ใหม่จาก Core Point ที่ยังไม่ถูกมอบหมาย

ข้อดีและข้อเสียของ DBSCAN

Tan et al. (2006) สรุปข้อดีของ DBSCAN ไว้ว่า: สามารถค้นพบ Cluster ที่มีรูปร่างซับซ้อนได้ ไม่ต้องกำหนดจำนวน Cluster ล่วงหน้า ทนทานต่อ Outlier และ Noise อย่างไรก็ตาม DBSCAN มีข้อเสียคือ ความไวต่อค่าพารามิเตอร์ ϵ และ MinPts ซึ่งต้องปรับค่าอย่างระมัดระวัง และมีปัญหากับข้อมูลที่มีความหนาแน่นไม่สม่ำเสมอ (Varying Density) โดย Hastie et al. (2009) แนะนำ OPTICS (Ordering Points To Identify the Clustering Structure) เป็นทางเลือกที่รองรับข้อมูลที่มีความหนาแน่นต่างกันได้ดีกว่า โดยสร้าง Reachability Plot เพื่อช่วยในการระบุ Cluster

8.4.2 แบบฝึกหัด

8.4.2.1 การตั้งค่า

```
import numpy as np
from sklearn.cluster import DBSCAN
from sklearn import metrics
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler

centers = [[1, 1], [-1, -1], [1, -1]]
X, labels_true = make_blobs(n_samples=750,
                           centers=centers,
                           cluster_std=0.4, random_state=0)
X = StandardScaler().fit_transform(X)
```

8.4.2.2 DBSCAN

```
db = DBSCAN(eps=0.3, min_samples=10).fit(X)
labels = db.labels_
```

```
n_clusters_ = len(set(labels)) - (1 if -1 in
labels else 0)
n_noise_ = list(labels).count(-1)

print('Estimated number of clusters: %d' %
n_clusters_) # 3
print('Estimated number of noise points: %d' %
n_noise_) # 18
print('Homogeneity: %0.3f' %
metrics.homogeneity_score(labels_true, labels))
# 0.953
print('Completeness: %0.3f' %
metrics.completeness_score(labels_true,
labels)) # 0.883
print('V-measure: %0.3f' %
metrics.v_measure_score(labels_true, labels)) #
0.917
print('Adjusted Rand Index: %0.3f' %
metrics.adjusted_rand_score(labels_true,
labels)) # 0.952
print('Silhouette Coefficient: %0.3f' %
metrics.silhouette_score(X, labels)) # 0.626
```

ผลลัพธ์: DBSCAN ค้นพบ 3 Cluster ได้สำเร็จ มี Noise 18 จุด และค่า Adjusted Rand Index สูงถึง 0.952 แสดงถึงการจับกลุ่มที่แม่นยำมาก

8.4.3 กรณีศึกษา

ดำเนินการ Density-Based Clustering โดยใช้ชุดข้อมูล Iris และแสดง DBSCAN พร้อมค่า eps และ min_samples หลายค่าเพื่อเปรียบเทียบผลลัพธ์

8.4.3.1 การตั้งค่า

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score,
adjusted_rand_score

data = load_iris()
X, y = data.data, data.target
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

8.4.3.2 DBSCAN

ทดสอบ DBSCAN ด้วยค่า $\epsilon = [0.2, 0.3, 0.4, 0.5]$ และ $\text{min_samples} = [2, 3, 4]$:

```
eps_values = [0.2, 0.3, 0.4, 0.5]
min_samples_values = [2, 3, 4]

fig, axes = plt.subplots(len(eps_values),
len(min_samples_values), figsize=(15, 15))
for i, eps in enumerate(eps_values):
    for j, min_samples in enumerate(min_samples_values):
        dbscan = DBSCAN(eps=eps,
min_samples=min_samples)
        dbscan_labels = dbscan.fit_predict(X_scaled)
        n_clusters_ = len(set(dbscan_labels)) - (1 if -
1 in dbscan_labels else 0)
        silhouette_avg = silhouette_score(X_scaled,
dbscan_labels)
        ari_score = adjusted_rand_score(y,
dbscan_labels)
        axes[i,j].scatter(X_scaled[:,0], X_scaled[:,1],
c=dbscan_labels, cmap='viridis')
        axes[i,j].set_title(f'eps={eps},
min={min_samples}\n')
```

```
f'Clusters:{n_clusters_},  
Sil:{silhouette_avg:.2f}')
```

```
plt.show()
```

8.5 Grid-Based Clustering

Grid-Based Clustering เป็นเทคนิคที่สร้างสรรค์ใน Unsupervised Machine Learning ที่แบ่งพื้นที่ Feature ออกเป็น Grid และระบุ Cluster ภายใน Grid Cell เหล่านั้น ส่วนนี้แนะนำวิธีการ Grid-Based Clustering โดยเฉพาะ STING และ OPTICS โดยใช้แพ็คเกจ Scikit-learn

Grid-Based Clustering
เหมาะอย่างยิ่งสำหรับชุดข้อมูลที่มีโครงสร้างเชิงพื้นที่หรือมีความหนาแน่นไม่สม่ำเสมอ โดยแบ่งพื้นที่ Feature ออกเป็น Grid ของ Cell และระบุ Cluster ภายใน Cell เหล่านี้

8.5.1 ทฤษฎี Grid-Based Clustering

Calinski-Harabasz Index

$$CH = \frac{\frac{SS_B}{K-1}}{\frac{SS_W}{n-K}}$$

โดยที่:

CH = Calinski-Harabasz Index ยิ่งสูงยิ่งดี

SSB = Sum of Squares Between clusters วัดการกระจายระหว่างกลุ่ม

SSW = Sum of Squares Within clusters วัดการกระจายภายในกลุ่ม

K = จำนวนกลุ่ม

n = จำนวนข้อมูลทั้งหมด

ใช้เลือกจำนวนกลุ่ม K ที่เหมาะสม: ค่า CH สูงสุดที่ K

ได้แสดงว่านั่นคือจำนวนกลุ่มที่ดีที่สุด

Grid-Based Clustering เป็นวิธีการจัดกลุ่มที่แบ่งพื้นที่ Feature ออกเป็น Grid Cell แล้วดำเนินการ Clustering บน Cell เหล่านั้นแทนที่จะดำเนินการกับจุดข้อมูลแต่ละจุดโดยตรง ทำให้มีความเร็วสูงและเหมาะสมกับชุดข้อมูลขนาดใหญ่

STING: Statistical Information Grid

STING (Statistical Information Grid) เป็นอัลกอริทึม Grid-Based Clustering ที่แบ่งพื้นที่ข้อมูลออกเป็น Rectangular Cell แบบลำดับชั้น โดยแต่ละ Cell จัดเก็บ Statistical Information เช่น ค่าเฉลี่ย (Mean) ส่วนเบี่ยงเบนมาตรฐาน (Standard Deviation) และการกระจายความน่าจะเป็น Tan et al. (2006) อธิบายว่า STING ทำงานจากบนลงล่าง (Top-Down) โดยเริ่มจาก Cell ระดับสูงสุดแล้วค้นหา Cell ที่มีความสำคัญ (Relevant Cell) ตามเกณฑ์ที่กำหนด จากนั้นขยายการค้นหาต่อไปใน Cell ระดับล่างที่เป็น Child ของ Cell ที่สำคัญ ข้อดีของ STING คือสามารถตอบคำถาม Query ได้หลากหลายโดยไม่ต้องประมวลผลใหม่ทั้งหมด

CLIQUE: Clustering In QUEst

CLIQUE เป็นอัลกอริทึม Subspace Clustering แบบ Grid-Based ที่ค้นหา Cluster ในมิติย่อย (Subspace) ของข้อมูล Tan et al. (2006) อธิบายว่า CLIQUE แบ่งแต่ละมิติออกเป็นช่วงเท่าๆ กัน (Intervals) จำนวน ζ ช่วง และนิยาม Unit (Grid Cell) เป็นจุดตัดของช่วงในแต่ละมิติ Unit ที่มีจำนวนจุดข้อมูลเกินเกณฑ์ความหนาแน่น τ จะถูกนับเป็น Dense Unit CLIQUE ค้นหา Dense Unit ทั้งหมดแล้วรวม Dense Unit ที่อยู่ติดกันเข้าเป็น Cluster

ข้อดีและข้อจำกัดของ Grid-Based Clustering

Tan et al. (2006) สรุปว่า Grid-Based Clustering มีประสิทธิภาพการคำนวณสูง เนื่องจากความซับซ้อนของเวลาขึ้นกับจำนวน Grid Cell ไม่ใช่จำนวนจุดข้อมูล

ทำให้เหมาะสำหรับชุดข้อมูลขนาดใหญ่ อย่างไรก็ตาม ผลลัพธ์ขึ้นอยู่กับข้อกำหนดขนาด Grid Cell อย่างมาก และอาจพลาด Cluster ที่อยู่ระหว่างขอบเขต Cell

8.5.2 แบบฝึกหัด

8.5.2.1 การตั้งค่า

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

X, y = make_blobs(n_samples=500, centers=4,
                  cluster_std=1.5, random_state=42)
plt.figure(figsize=(8, 6))
plt.scatter(X[:,0], X[:,1], c='b', s=50)
plt.title('Dummy Grid-like Dataset')
plt.show()
```

8.5.2.2 STING

ใช้ KMedoids เพื่อจำลอง STING Clustering (Statistical Information Grid) ซึ่งแบ่งชุดข้อมูลออกเป็น Grid แบบลำดับชั้น:

```
!pip install scikit-learn-extra
from sklearn_extra.cluster import KMedoids

sting = KMedoids(n_clusters=4, random_state=42)
sting_labels = sting.fit_predict(X)
print(f'STING          Cluster          Assignments:
      {sting_labels}')

plt.scatter(X[:,0], X[:,1], c=sting_labels,
            cmap='viridis', s=50)
plt.title('STING Clustering')
plt.show()
```

8.5.2.3 CLIQUE

```
!pip install pyclustering
from pyclustering.cluster.clique import clique,
clique_visualizer

intervals = 20 # จำนวน Cell ใน Grid แต่ละมิติ
threshold = 3 # ความหนาแน่นขั้นต่ำ
clique_instance = clique(X.tolist(), intervals,
threshold)
clique_instance.process()

clusters = clique_instance.get_clusters()
noise = clique_instance.get_noise()
cells = clique_instance.get_cells()
print('Amount of clusters:', len(clusters)) # 4

clique_visualizer.show_grid(cells, X.tolist())
clique_visualizer.show_clusters(X.tolist(),
clusters, noise)
```

8.5.3 กรณีศึกษา

ใช้ชุดข้อมูล Two Moons ซึ่งประกอบด้วย Cluster รูปพระจันทร์เสี้ยวสองอัน เพื่อแสดงและเปรียบเทียบ STING, OPTICS และ DBSCAN

8.5.3.1 การตั้งค่า

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=500, noise=0.05,
random_state=42)
```

```
plt.scatter(X[:,0], X[:,1], c='b', s=50)
plt.title('Two Moons Dataset')
plt.show()
```

8.5.3.2 STING Clustering

STING แบ่งชุดข้อมูลออกเป็น Grid แบบลำดับขั้น แล้วจัดกลุ่ม Cell ตามคุณลักษณะทางสถิติ:

```
from sklearn_extra.cluster import KMedoids
from sklearn.metrics import adjusted_rand_score

sting = KMedoids(n_clusters=2, random_state=42)
sting_labels = sting.fit_predict(X)
ari_sting = adjusted_rand_score(y, sting_labels)
print(f'STING ARI Score: {ari_sting:.4f}') #
0.2816
```

สรุป: STING ไม่เหมาะกับชุดข้อมูล Two Moons เนื่องจาก Density-Based Clustering ทำได้ดีกว่า

8.5.3.3 OPTICS Clustering

OPTICS เป็นวิธี Density-Based ที่สร้าง Reachability Plot เพื่อระบุ Cluster ที่มีความหนาแน่นต่างกัน:

```
from sklearn.cluster import OPTICS

optics = OPTICS(min_samples=10, xi=0.15,
min_cluster_size=0.15)
optics_labels = optics.fit_predict(X)
ari_optics = adjusted_rand_score(y,
optics_labels)
print(f'OPTICS ARI Score: {ari_optics:.4f}') #
1.0000
```


สรุป: OPTICS ให้ ARI Score = 1.0 แสดงว่าจัดกลุ่มได้อย่างสมบูรณ์แบบ

นอกจากนี้ยังทดลอง DBSCAN บนชุดข้อมูลเดียวกัน:

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.15, min_samples=10)
dbscan_labels = dbscan.fit_predict(X)
ari_dbscan = adjusted_rand_score(y,
dbscan_labels)
print(f'DBSCAN ARI Score: {ari_dbscan:.4f}') #
1.0000
```

สรุป: ทั้ง OPTICS และ DBSCAN ให้ ARI Score = 1.0 บนชุดข้อมูล Two Moons แสดงว่าวิธี Density-Based เหมาะสมกับข้อมูลที่มีรูปทรงซับซ้อน

8.6 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) เป็นเทคนิคลด Dimension ขั้นพื้นฐานที่ใช้กันอย่างแพร่หลายใน Machine Learning และการวิเคราะห์ข้อมูล ส่วนนี้แนะนำ PCA ซึ่งรวมทั้งวิธีแบบกำหนดจำนวน Component และแบบกำหนดสัดส่วน Variance โดยใช้แพ็คเกจ Scikit-learn

PCA แปลง Feature เดิมให้เป็นชุดตัวแปรมุมฉากใหม่ที่เรียกว่า Principal Component ใช้สำหรับการบีบอัดข้อมูล การแสดงภาพ และการเลือก Feature — วิธีเลือก Component มีสองแบบ คือ กำหนดจำนวน ($n_components=k$) และกำหนดสัดส่วน Variance (เช่น 0.95)

8.6.1 ทฤษฎี Principal Component Analysis (PCA)

Principal Component Analysis (PCA) เป็นเทคนิคการลดมิติ (Dimensionality Reduction) ที่ใช้กันอย่างแพร่หลายที่สุดในวิทยาศาสตร์ข้อมูล Bishop (2006) อธิบายว่า PCA ทำการฉาย (Project) ข้อมูลจากพื้นที่มิติสูงไปยังพื้นที่มิติต่ำ โดยรักษาความแปรปรวน (Variance) ของข้อมูลให้มากที่สุดเท่าที่เป็นไปได้

หลักการทางคณิตศาสตร์ของ PCA

Hastie et al. (2009) อธิบายว่า PCA หา Best Linear Approximation ของข้อมูลในทุก Rank $q \leq p$ โดยพิจารณาโมเดลเชิงเส้น $f(\lambda) = \mu + V_q \lambda$ ซึ่ง V_q เป็นเมทริกซ์ $p \times q$ ที่มี q Orthogonal Unit Vectors เป็นคอลัมน์ การหาค่า V_q ที่ดีที่สุดทำได้โดย Singular Value Decomposition (SVD) ของเมทริกซ์ข้อมูล $X = UDV^T$ โดย Principal Components ของ X คือคอลัมน์ของ UD และ V_q ที่ดีที่สุดประกอบด้วย q คอลัมน์แรกของ V

Explained Variance Ratio

Bishop (2006) อธิบายว่า Variance ที่ถูกอธิบายโดย Principal Component แต่ละตัว (Explained Variance Ratio) คำนวณจาก Eigenvalue ที่สัมพันธ์กัน ยิ่ง Eigenvalue สูงยิ่งมีความสำคัญมาก การเลือกจำนวน Component โดยกำหนดสัดส่วน Variance ขั้นต่ำ (เช่น 95%) หมายความว่าเลือก Component น้อยที่สุดที่ทำให้ผลรวม Explained Variance Ratio เกินค่าที่กำหนด

การเลือกจำนวน Principal Components

Hastie et al. (2009) แนะนำวิธีการเลือกจำนวน Principal Component ที่เหมาะสมโดยดูจาก Scree Plot ซึ่งแสดงค่า Eigenvalue ตามลำดับ จุด “ข้อศอก” (Elbow Point) ในกราฟบ่งบอกถึงจำนวน Component ที่เหมาะสม นอกจากนี้ยังสามารถเลือกตาม Cumulative Explained Variance โดยทั่วไปนักปฏิบัติมักเลือกจำนวน Component ที่ให้ค่า Cumulative Variance ถึง 80-95% Tan et al. (2006) เน้นว่า PCA มีประโยชน์ในการแก้ปัญหา Curse of

Dimensionality ซึ่งเป็นปรากฏการณ์ที่การวิเคราะห์ข้อมูลในมิติสูงยากขึ้นอย่างมากเนื่องจากความหนาแน่นของข้อมูลลดลงอย่างรวดเร็วเมื่อจำนวนมิติเพิ่มขึ้น

ข้อจำกัดของ PCA

Bishop (2006) กล่าวว่า PCA เป็นเทคนิค Linear ทำให้ไม่สามารถจับความสัมพันธ์ที่ซับซ้อน (Nonlinear) ในข้อมูลได้อย่างเต็มที่ นอกจากนี้ PCA ยังไวต่อ Outlier และต้องทำการ Standardize ข้อมูลก่อนในกรณีที่แต่ละ Feature มีหน่วยหรือสเกลต่างกัน สำหรับข้อมูลที่มีความสัมพันธ์ Nonlinear อาจพิจารณาใช้ Kernel PCA หรือ Nonlinear Dimensionality Reduction Methods อื่นๆ แทน

8.6.2 แบบฝึกหัด

8.6.2.1 ทดลองที่ 1: ใช้ชุดข้อมูล Digits (100% Feature)

โหลดและประมวลผลชุดข้อมูล Digits ซึ่งมี 1,797 ตัวอย่าง 64 Feature (pixel 8×8):

```
from sklearn.datasets import load_digits
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

dataset = load_digits()
dataset.data.shape # (1797, 64)

X = pd.DataFrame(dataset.data,
                  columns=dataset.feature_names)
y = dataset.target
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
random_state=30)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
model.score(X_test, y_test) # 0.9722
```

สรุป: ด้วยข้อมูล 100% (64 Feature) ได้ความแม่นยำ 97.2%

8.6.2.2 ทดลองที่ 2: PCA 0.95 (เก็บ 95% ของ Variance)

```
from sklearn.decomposition import PCA
```

```
pca = PCA(0.95)
X_pca = pca.fit_transform(X)
X_pca.shape # (1797, 40)
pca.n_components_ # 40
```

```
X_train_pca, X_test_pca, y_train, y_test =
train_test_split(X_pca, y, test_size=0.2,
random_state=30)
model = LogisticRegression(max_iter=1000)
model.fit(X_train_pca, y_train)
model.score(X_test_pca, y_test) # 0.9639
```

สรุป: ด้วยข้อมูล 95% (40 Feature) ได้ความแม่นยำ 96.4%

8.6.2.3 ทดลองที่ 3: PCA 0.80 (เก็บ 80% ของ Variance)

```
pca = PCA(0.80)
X_pca = pca.fit_transform(X)
X_pca.shape # (1797, 21)
model.score(X_test_pca, y_test) # 0.9472
```

สรุป: ด้วยข้อมูล 80% (21 Feature) ได้ความแม่นยำ 94.7%

8.6.2.4 ทดลองที่ 4: PCA 2 Feature

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
pca.explained_variance_ratio_ # [0.120, 0.096]
model.score(X_test_pca, y_test) # 0.5667
```

สรุป: ด้วยเพียง 2 Feature ได้ความแม่นยำ 56.7% ลดลงอย่างมาก เนื่องจากข้อมูลมากกว่า 78% สูญเสียไป

8.6.2.5 ทดลองที่ 5: PCA 10 Feature

```
pca = PCA(n_components=10)
X_pca = pca.fit_transform(X)
X_pca.shape # (1797, 10)
model.score(X_test_pca, y_test) # 0.8806
```

สรุป: ด้วย 10 Feature ได้ความแม่นยำ 88.1%

8.6.3 กรณีศึกษา

8.6.3.1 ทดลองที่ 1: ใช้ชุดข้อมูล Wine (100% Feature)

โหลดและประมวลผลชุดข้อมูล Wine ซึ่งมี 178 ตัวอย่าง 13 Feature:

```
from sklearn.datasets import load_wine

dataset = load_wine()
dataset.data.shape # (178, 13)

X = pd.DataFrame(dataset.data,
                  columns=dataset.feature_names)
```

```
y = dataset.target
scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.2,
random_state=30)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
model.score(X_test, y_test) # 0.9722
```

สรุป: ด้วยข้อมูล 100% (13 Feature) ได้ความแม่นยำ 97.2%

8.6.3.2 ทดลองที่ 2: PCA 0.95

```
pca = PCA(0.95)
X_pca = pca.fit_transform(X)
X_pca.shape # (178, 10)
model.score(X_test_pca, y_test) # 0.9722
```

สรุป: ด้วยข้อมูล 95% (10 Feature) ได้ความแม่นยำ 97.2% — ดีพอๆ กับใช้ข้อมูลทั้งหมด

8.6.3.3 ทดลองที่ 3: PCA 0.80

```
pca = PCA(0.80)
X_pca = pca.fit_transform(X)
X_pca.shape # (178, 5)
model.score(X_test_pca, y_test) # 0.9722
```

สรุป: ด้วยข้อมูล 80% (5 Feature) ยังได้ความแม่นยำ 97.2%

8.6.3.4 ทดลองที่ 4: PCA 2 Feature

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
```

```
model.score(X_test_pca, y_test) # 0.9722
```

สรุป: ด้วยเพียง 2 Feature ยังได้ความแม่นยำ 96.2% — ชุดข้อมูล Wine ลด Dimension ได้อย่างมีประสิทธิภาพมาก

8.6.3.5 ทดลองที่ 5: PCA 1 Feature

```
pca = PCA(n_components=1)
X_pca = pca.fit_transform(X)
pca.explained_variance_ratio_ # [0.362]
model.score(X_test_pca, y_test) # 0.8889
```

สรุป: ด้วยเพียง 1 Feature ยังได้ความแม่นยำ 88.9%

8.7 การเปรียบเทียบวิธีการ Clustering

ในส่วนนี้ดำเนินการกรณีศึกษาเชิงลึกเพื่อสำรวจและเปรียบเทียบประสิทธิภาพของวิธีการ Clustering ต่างๆ ที่ได้แนะนำในบทนี้ โดยใช้ชุดข้อมูลชุดเดียวกัน

กรณีศึกษานี้มุ่งประเมินวิธี Clustering สามประเภท ได้แก่ Partitioning Methods, Hierarchical Methods และ Density-Based Methods โดยใช้ Silhouette Score เป็นตัวชี้วัดหลัก เพื่อช่วยในการเลือกเทคนิคที่เหมาะสมที่สุดสำหรับงาน Clustering ที่กำหนด

8.7.1 กรณีศึกษา

ดำเนินการ Clustering บนชุดข้อมูล Penguins โดยแสดงวิธี Clustering สามประเภท แล้วเปรียบเทียบด้วย Silhouette Score

8.7.1.1 การตั้งค่า

โหลดและสำรวจชุดข้อมูล Penguins:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

penguins = sns.load_dataset('penguins')
print(penguins.head())
print(penguins.info())
print(penguins.describe())

penguins = penguins.dropna()
penguins = pd.get_dummies(penguins,
                           columns=['species', 'island', 'sex'],
                           drop_first=True)
X = penguins.drop('species_Chinstrap', axis=1)
```

8.7.1.2 Clustering ด้วย Partitioning Methods

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X)

plt.scatter(X['bill_length_mm'],
            X['bill_depth_mm'], c=kmeans_labels,
            cmap='viridis', s=50)
plt.title('K-means Clustering')
plt.xlabel('Bill Length (mm)')
plt.ylabel('Bill Depth (mm)')
plt.show()

silhouette_kmeans = silhouette_score(X,
kmeans_labels)
```

```
print(f'Silhouette Score (K-means):
      {silhouette_kmeans:.4f}') # 0.5752
```

8.7.1.3 Clustering ด้วย Hierarchical Methods

```
from sklearn.cluster import
AgglomerativeClustering

agglomerative =
AgglomerativeClustering(n_clusters=3)
agglomerative_labels =
agglomerative.fit_predict(X)

plt.scatter(X['bill_length_mm'],
            X['bill_depth_mm'], c=agglomerative_labels,
            cmap='viridis', s=50)
plt.title('Agglomerative Clustering')
plt.show()

silhouette_agglomerative = silhouette_score(X,
agglomerative_labels)
print(f'Silhouette Score (Agglomerative):
      {silhouette_agglomerative:.4f}') # 0.5164
```

8.7.1.4 Clustering ด้วย Density-Based Methods

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=20, min_samples=2)
dbscan_labels = dbscan.fit_predict(X)

plt.scatter(X['bill_length_mm'],
            X['bill_depth_mm'], c=dbscan_labels,
            cmap='viridis', s=50)
plt.title('DBSCAN Clustering')
plt.show()
```

```
silhouette_dbscan      =      silhouette_score(X,
silhouette_labels)
print(f'Silhouette      Score      (DBSCAN) :
{silhouette_dbscan:.4f}') # 0.6150
```

8.7.1.5 เปรียบเทียบผลลัพธ์และสรุป

```
silhouette_scores      =      [silhouette_kmeans,
silhouette_agglomerative, silhouette_dbscan]
method_names      =      ['K-means',      'Agglomerative',
'DBSCAN']
```

```
for i, score in enumerate(silhouette_scores):
    print(f'Silhouette Score ({method_names[i]}):
{score:.4f}')
```

```
# Silhouette Score (K-means): 0.5752
# Silhouette Score (Agglomerative): 0.5164
# Silhouette Score (DBSCAN): 0.6150
```

สรุป: จากการเปรียบเทียบด้วย Silhouette Score บนชุดข้อมูล Penguins พบว่า DBSCAN ให้คะแนนสูงสุด (0.6150) รองลงมาคือ K-Means (0.5752) และ Agglomerative Clustering (0.5164) ตามลำดับ การเลือกวิธี Clustering ที่เหมาะสมขึ้นอยู่กับลักษณะของข้อมูลและเป้าหมายของการวิเคราะห์